Klosnowo — Documentation technique

Version V1 — moteur de jeu en C + mode local 2 joueurs (terminal)

Projet ENSISA Immersion 2026 — jeu de plateau Klosnowo (inspiré de Tablut/Hnefatafl)

Ce document explique, fichier par fichier et fonction par fonction, comment le programme est construit : les structures de données, les règles du jeu telles qu'implémentées, et le fonctionnement du mode de jeu en terminal (affichage, animations, saisie des coups).

1. Vue d'ensemble du projet

Klosnowo est un jeu de plateau à deux joueurs (BLEU et ROUGE), inspiré des jeux de type Tafl (Tablut / Hnefatafl), sur un plateau rectangulaire de 11 colonnes sur 7 lignes. Chaque joueur dispose d'un roi, de 9 soldats et d'une cité. Le but est soit d'amener son roi sur la cité adverse (conquête), soit de capturer le roi adverse (exécution), soit — si la partie dure 64 demi-tours — d'avoir le meilleur score (cases contrôlées + soldats vivants).

Le projet est pensé comme une architecture modulaire en C :

- **board.h / board.c** : le moteur de jeu pur (règles, plateau, coups, captures, score) — **terminé**.
- **main.c** : le programme jouable, pour l'instant en mode local (2 joueurs humains sur le même clavier, dans le terminal) — **terminé**.
- **network.h / network.c** : communication réseau (sockets TCP) pour jouer à distance — **pas encore fait**.
- **ai.h / ai.c** : intelligence artificielle (Minimax + élagage Alpha-Bêta) — **pas encore fait**.

Cette version V1 couvre donc entièrement le moteur de jeu et un mode de jeu local complet, animé et jouable au clavier dans le terminal.

2. Arborescence des fichiers

```
klosnowo/
|-- Makefile                 compilation du projet
|-- include/
|   `-- board.h               declarations : types, structures, fonctions du moteur
`-- src/
    |-- board.c               implementation du moteur de jeu (regles, plateau, captures)
    `-- main.c                programme jouable en terminal (mode local 2 joueurs)
```

3. include/board.h — Interface du moteur de jeu

Ce fichier ne contient aucune logique : uniquement les *déclarations* (constantes, types, structures, signatures de fonctions) que board.c implémente et que main.c utilise. C'est le "contrat" du moteur de jeu.

3.1 Constantes

- **BOARD_WIDTH** (11) et **BOARD_HEIGHT** (7) : dimensions du plateau.
- **SOLDIERS_PER_PLAYER** (9) : nombre de soldats par camp.
- **MAX_HALF_MOVES** (64) : limite de demi-tours (32 coups par joueur) avant de trancher au score.
- **MAX_LEGAL_MOVES** (512) et **MAX_CAPTURES_PER_MOVE** (8) : tailles de tableaux fixes (pas d'allocation dynamique nécessaire, le plateau est petit : 77 cases).

3.2 Types énumérés

`Player`

PLAYER_NONE, PLAYER_BLUE, PLAYER_RED. Représente un camp, ou l'absence de camp (case neutre, case sans pièce).

`PieceType`

PIECE_NONE, PIECE_SOLDIER, PIECE_KING. Le type d'occupant d'une case.

`GameStatus`

État de la partie : STATUS_ONGOING (en cours), puis une valeur par condition de fin : *_WINS_CONQUEST, *_WINS_EXECUTION, *_WINS_SCORE, ou STATUS_DRAW (égalité au score, cas rare).

`CaptureType`

CAPTURE_LINCA (prise en sandwich) ou CAPTURE_SEULTOU (prise en poussant, sans soutien).

3.3 Structures de données

`Position { x, y }`

Une coordonnée sur le plateau (colonne x, ligne y), stockée en int8_t pour rester compact.

`Move { from, to }`

Un coup : case de départ et case d'arrivée.

`Capture { position, type }`

Une capture individuelle : la case qui vient d'être vidée, et le type de prise (Linca ou Seultou).

`MoveResult { legal, capture_count, captures[], status_after }`

Le résultat complet de l'application d'un coup : est-il légal, combien de captures ont eu lieu et lesquelles (une seule pièce en mouvement peut déclencher plusieurs captures en chaîne), et l'état de la partie juste après.

`MoveList { moves[], count }`

Une liste de coups à taille fixe (prévue pour un futur générateur de coups légaux, utile à l'IA).

`Cell { piece, piece_owner, controlled_by, is_city, is_barricade }`

Une case du plateau : quelle pièce l'occupe (et son propriétaire), quel camp *contrôle* la case (indépendamment de l'occupation actuelle), si c'est une cité, si une barricade y est posée.

`GameState`

L'état complet d'une partie : le plateau (tableau de Cell), le joueur actif, le statut, la position du roi/de la cité/de la barricade de chaque camp, le nombre de soldats vivants par camp, et le compteur de demi-tours.

3.4 Fonctions déclarées (implémentées dans board.c)

`void board_init(GameState *state)`

Initialise une nouvelle partie : plateau vide, cités, placement de départ du roi et des 9 soldats de chaque camp, contrôle initial, tour au joueur BLEU, aucune barricade posée.

`bool board_place_barricade(GameState *state, Player player, Position pos)`

Pose la barricade de `player` en `pos` si la position est valide (voir 4.3). Retourne false sans rien modifier sinon.

`const char *board_barricade_error(...)`

Comme ci-dessus, mais renvoie NULL si valide ou un message d'erreur précis sinon (utilisé par main.c pour afficher pourquoi un choix est refusé).

`bool board_in_bounds(Position pos)`

Vrai si la position est dans les limites du plateau.

`bool board_is_legal_move(...) / const char *board_move_error(...)`

Même principe que pour la barricade : valide un déplacement (ligne orthogonale, chemin libre, arrivée légale), avec une variante qui explique pourquoi un coup est refusé.

`MoveResult board_apply_move(GameState *state, Move move)`

Applique un coup déjà légal : déplace la pièce, met à jour le contrôle des cases, résout toutes les captures déclenchées, vérifie les conditions de victoire, avance le compteur de demi-tours et passe la main.

`int board_compute_score(const GameState *state, Player player)`

Cases contrôlées + soldats vivants (le roi ne compte pas) pour ce joueur.

`GameStatus board_check_end_condition(const GameState *state)`

Renvoie l'état courant de la partie.

`Player board_opponent(Player p)`

Renvoie l'adversaire d'un joueur (fonction "inline", très légère).

`void board_print(const GameState *state)`

Affiche le plateau dans le terminal, avec des couleurs ANSI.

4. src/board.c — Implémentation du moteur de jeu

C'est le cœur du jeu : toutes les règles y sont implémentées. Les fonctions marquées "statiques" (`static`) sont des détails internes, invisibles depuis les autres fichiers.

4.1 Fonctions utilitaires internes

`pos_make(x, y) / pos_equal(a, b)`

Construisent et comparent des Position. Simples raccourcis pour éviter de répéter des conversions de type partout.

`cell_at(state, p) / cell_at_c(state, p)`

Renvoient un pointeur vers la case à la position `p` (version modifiable et version lecture seule). Toutes les autres fonctions passent par ces deux-là pour accéder au plateau, ce qui centralise l'indexation `board[y][x]`.

`bool board_in_bounds(Position pos)`

Vérifie 0 ≤ x < BOARD_WIDTH et 0 ≤ y < BOARD_HEIGHT.

4.2 Placement initial (board_init)

Le roi et les 9 soldats de chaque camp forment un petit bloc en forme d'escalier dans leur coin de départ. Ce bloc est décrit une seule fois, pour le joueur BLEU (coin haut-gauche), dans le tableau `CORNER_BLOCK` :

```
{0,2}  {0,3}
       {1,1}=ROI {1,2} {1,3}
{2,0} {2,1} {2,2}
{2,0}... {3,0} {3,1}
```

Hypothèse importante : le document de règles fourni illustre ce placement sur un plateau 9x9 "pédagogique", pas sur le vrai plateau 11x7. En l'absence d'un schéma officiel pour 11x7, ce même bloc est réutilisé tel quel dans le coin de chaque joueur — c'est un choix raisonnable mais **à confirmer** si un document plus précis existe.

`place_corner(state, player, mirror)`

Place ce bloc pour un camp donné. Pour le camp ROUGE, `mirror=true` : chaque position est retournée symétriquement (x → largeur-1-x, y → hauteur-1-y) pour obtenir le coin bas-droit opposé.

`board_init(state)`

Vide entièrement le plateau, place les deux cités (coins (0,0) et (largeur-1,hauteur-1)), place les deux blocs de départ via place_corner, initialise les compteurs (9 soldats vivants par camp, 0 demi-tour, aucune barricade), et donne le trait au BLEU.

4.3 Règles de la barricade

Avant le premier coup, chaque joueur pose une barricade : une case bloquante, sans propriétaire, qui ne peut ni être traversée ni recevoir de pièce.

`diagonal_value(pos) et DIAGONAL_REFERENCE`

La diagonale du plateau correspond aux cases où x+y est constant (c'est la diagonale visible sur les schémas de règles). Le seuil qui coupe le rectangle en deux moitiés symétriques est (largeur-1 + hauteur-1) / 2, soit 8 pour un plateau 11x7. Une case avec x+y < 8 est du côté BLEU, x+y > 8 du côté ROUGE, x+y = 8 est sur la diagonale (interdite aux deux joueurs).

`is_adjacent_to_camp(state, player, pos)`

Vérifie les 8 cases voisines de `pos` : si l'une d'elles contient une pièce de `player` ou est sa cité, la position est trop proche du camp de départ et donc refusée.

`validate_barricade(...) / board_barricade_error(...)`

Enchaînent les vérifications (hors plateau, cité, déjà occupée, déjà une barricade, mauvais côté de la diagonale, adjacente au camp) et renvoient un message précis dès la première qui échoue, ou NULL si tout est bon. board_barricade_error() ajoute deux vérifications de tour : la barricade du joueur n'est pas déjà posée, et le BLEU doit avoir posé la sienne avant que le ROUGE ne puisse poser la sienne.

`board_place_barricade(...)`

Pose réellement la barricade si board_barricade_error() renvoie NULL.

4.4 Règles de déplacement

`validate_move(...) / board_move_error(...) / board_is_legal_move(...)`

Vérifient, dans l'ordre : que les cases existent, que départ ≠ arrivée, que le déplacement est strictement horizontal ou vertical (jamais en diagonale), qu'il y a bien une pièce du joueur en case de départ, puis parcourent chaque case du chemin : une case intermédiaire occupée ou une barricade bloque le coup (pas de saut), et la case d'arrivée doit être libre, sans barricade, et ne peut être une cité que si la pièce déplacée est le roi.

4.5 Application d'un coup et captures (board_apply_move)

Cette fonction fait tout le travail une fois un coup validé :

- Déplace la pièce (case de départ vidée, case d'arrivée occupée).
- Met à jour le *contrôle* de toutes les cases traversées et de la case d'arrivée (sauf les cités, jamais contrôlées) — la case quittée reste contrôlée par le joueur, on ne touche pas à son champ controlled_by.
- Si la pièce déplacée est le roi et qu'il atterrit sur la cité adverse : victoire par **conquête**.
- Cherche des captures dans les 4 directions autour de la case d'arrivée (seule la pièce qui vient de bouger peut capturer, plusieurs captures peuvent se déclencher sur un même coup) :

Pour chaque direction où une pièce ennemie est immédiatement adjacente, on regarde la case juste après (*beyond*) :

- hors du plateau, ou vide et sans barricade → **Seultou** (pas de soutien) : capture.
- occupée par un allié de l'attaquant → **Linca** (prise en sandwich) : capture.
- occupée par un allié du défenseur, ou par une barricade → protégée, pas de capture. La barricade agit ainsi comme un allié du joueur *passif* (celui qui ne vient pas de jouer) : elle ne complète jamais une prise pour l'attaquant, et elle fait toujours office de soutien contre un Seultou.

Chaque pièce capturée est retirée et sa case redevient neutre (occupation ET contrôle). Si la pièce capturée est un roi, la partie se termine immédiatement par **exécution**. Enfin, le compteur de demi-tours est incrémenté ; s'il atteint la limite (64) et que la partie est encore en cours, le score des deux joueurs est comparé pour déterminer une victoire au score ou une égalité. Si la partie continue, la main passe à l'adversaire.

4.6 Score et affichage

`board_compute_score(state, player)`

Parcourt tout le plateau pour compter les cases contrôlées par `player`, et y ajoute le nombre de soldats encore en vie.

`board_check_end_condition(state)`

Renvoie simplement l'état déjà calculé par board_apply_move.

`print_cell(state, p) / board_print(state)`

Affichent une case (barricade, cité, pièce colorée selon le camp, ou point representant une case contrôlée/neutre) puis tout le plateau avec les numéros de colonnes et de lignes, en utilisant des codes couleur ANSI (texte bleu/rouge dans le terminal).

5. src/main.c — Programme jouable (mode local, 2 joueurs)

Ce fichier ne connaît aucune règle du jeu : il se contente d'appeler les fonctions de board.h et de gérer l'affichage terminal, la lecture au clavier, et quelques animations. Les deux joueurs jouent chacun leur tour sur le même clavier.

5.1 Affichage et texte

`player_name(p) / player_color(p)`

Renvoient "BLEU"/"ROUGE" et le code couleur ANSI correspondant.

`clear_screen()`

Envoie les séquences ANSI qui replacent le curseur en haut à gauche et effacent tout l'écran, pour que chaque tour s'affiche sur un écran "propre" plutôt que de s'accumuler.

`print_legend() / print_help() / print_scores()`

Affichent respectivement le rappel des symboles (Kb/Kr, sb/sr, Cb/Cr, ##), la liste des commandes disponibles, et les scores courants des deux joueurs.

5.2 Lecture des commandes

`read_line(buf, size)`

Lit une ligne au clavier (fgets) et retire le retour à la ligne final. Renvoie false si l'utilisateur ferme l'entrée (Ctrl-D).

`to_lower_inplace(s) / normalize_separators(s)`

Mettent une commande en minuscules pour la comparer facilement ("quitter", "aide"...), et remplacent les virgules/points-virgules par des espaces pour accepter "1,2" aussi bien que "1 2".

5.3 Introduction interactive

`print_intro()`

Affiche, au tout début du programme, un tutoriel en 3 étapes (barricade, déplacement, conditions de victoire) avec des exemples de commandes concrets, et attend que l'utilisateur appuie sur Entrée avant de continuer.

5.4 Animations

Trois animations rendent la partie plus vivante que du simple texte qui défile :

`animate_move(state, move)`

Fait glisser une pièce case par case à l'écran. Elle travaille sur une *copie complète* de l'état de la partie (une simple copie de structure, sans pointeur, donc sans risque) : le vrai board_apply_move n'est appelé qu'après, une fois l'animation terminée. À chaque case intermédiaire, l'écran est effacé et redessiné avec un court délai (120 ms).

`cell_term_row(y) / cell_term_col(x) / move_cursor(row, col)`

Calculent la position exacte (ligne, colonne) d'une case du plateau *dans le terminal*, à partir du format précis utilisé par board_print (ligne d'en-tête, puis 3 caractères par case à partir de la colonne 5). move_cursor y déplace directement le curseur avec la séquence ANSI correspondante.

`flash_captures(result, captured_owner)`

Fait clignoter en vidéo inversée chaque case capturée par le coup qui vient d'être joué, directement par-dessus le plateau déjà affiché (pas besoin de tout redessiner), avec un bip sonore (caractère '\a').

`flash_banner(text, color)`

Fait clignoter une ligne de texte (utilisée pour l'annonce de victoire en fin de partie) en alternant couleur normale et grisée.

5.5 Déroulement d'une partie

`prompt_barricade(state, player)`

Boucle de saisie de la barricade d'un joueur : lit une position, affiche le message précis de board_barricade_error() si elle est refusée (sans effacer l'écran, pour que le message reste visible), et continue jusqu'à un choix valide.

`print_move_result(result) / print_final_status(state)`

Affichent la liste des captures d'un coup (type + position), et à la fin de la partie, le plateau final, les scores, et l'annonce de victoire adaptée (conquête, exécution, score ou égalité) via flash_banner().

`int main(void)`

Assemble tout : initialise la partie (board_init), affiche l'introduction, fait poser les deux barricades (BLEU puis ROUGE), puis boucle tant que la partie est en cours : affiche le plateau et le score, lit une commande ('aide', 'plateau', 'quitter', ou un coup 'x1 y1 x2 y2'), valide le coup avec board_move_error(), l'anime, l'applique avec board_apply_move(), déclenche l'animation de capture si besoin, puis recommence pour l'autre joueur. À la fin, affiche l'écran de victoire.

6. Makefile — Compilation

Compile les deux fichiers source avec des avertissements stricts activés (-Wall -Wextra -Wpedantic), et produit un exécutable nommé `klosnowo`.

```
cd klosnowo
make run      # compile (si necessaire) puis lance le jeu
make          # compile seulement
make clean    # supprime l'executable compile
```

7. Comment jouer (rappel)

- Au lancement : lire le tutoriel, appuyer sur Entrée.
- Phase de barricade : BLEU tape "x y" en premier, puis ROUGE voit le choix de BLEU et tape le sien.
- Ensuite, à chaque tour : taper "x1 y1 x2 y2" pour déplacer la pièce de (x1,y1) vers (x2,y2), ou "plateau" / "aide" / "quitter".
- Coordonnées 0-indexées : x = colonne (affichée en haut), y = ligne (affichée à gauche), y=0 en haut.

8. Hypothèses et limites connues de cette version

- Le placement initial du bloc roi+soldats et la formule de diagonale de la barricade sont une généralisation du schéma pédagogique 9x9 du document de règles vers le vrai plateau 11x7 : le document ne fournit pas de schéma dédié à 11x7. À confirmer si un document plus précis existe.
- Aucune limite de temps par coup n'est imposée : chaque joueur réfléchit le temps qu'il veut.
- Le réseau (network.h/.c) et l'intelligence artificielle (ai.h/.c) ne sont pas encore développés : seul le mode local à deux joueurs humains, sur la même machine, est jouable pour l'instant.

9. Prochaines étapes

- **network.h / network.c** : sockets TCP (client/serveur), protocole texte simple du type "MOVE x1 y1 x2 y2\n" / "BARRICADE x y\n", pour jouer à deux sur deux machines différentes.
- **ai.h / ai.c** : génération des coups légaux, fonction d'évaluation (différentiel de soldats, de territoire, proximité du roi de la cité adverse), recherche Minimax avec élagage Alpha-Bêta à profondeur réglable, pour un adversaire artificiel.